

Scalability Roadmap: Partitioning and Long Term Architecture

Author: Justin Hunter **Date:** 2026-05-08 **Audience:** Engineering team, engineering leadership, and architecture review **Companion documents:** Program Charter, Query Store Triage and Refactor Program plan, Maintenance Operations Plan

1. Purpose

This is the long arc document. The other plans in the program describe what we are recommending in the next two quarters to relieve current pressure. This one describes what we are recommending over the next two years to make the architecture scale gracefully as we accumulate data and add MFC sites.

The headline is that the highest-volume tables in the PharmAssist schema are growing in a way that the current architecture will not absorb gracefully. Even after every refactor recommendation and every maintenance recommendation has been deployed, the underlying issue remains: a table holding hundreds of millions of rows where the active working set is the most recent sixty days does not scale linearly under any operation that touches the older data. Index seeks remain reasonable. Index scans, statistics rebuilds, fragmentation reconciliation, and bulk deletes all degrade as the table grows.

Table partitioning is the single most consequential architectural change available to us, and the partitioning POC modules already lay out the migration approach, the indexing strategy, the switch operation patterns, and the monitoring. The roadmap below is the recommendation for moving from POC to production.

This document is written from the same position as the other program documents. The MFC DBA team and I produce the recommendations and the supporting analysis. The iA DBA team, the iA software development teams, and the application teams that own the consumers of the partitioning candidate tables are the parties whose review, validation, and execution capacity determines when any of these recommendations actually deploy. The partitioning rollout in particular requires application-team coordination beyond the standard refactor pipeline, because partitioned tables can interact with application code in ways that unpartitioned tables do not. The phasing in section 7 reflects what we will produce on the analytical side. The production rollout cadence is for the iA team and the application teams to commit, separately.

2. The Growth Problem

The data velocity and capacity script in the diagnostic toolkit produces per-table growth rates per site. The procedure-level numbers in the Query Store Triage and Refactor Program plan show how much I/O these tables are absorbing today. The per-table picture is similar in shape: a small number of tables (`EvtEvent` , `OrderHistory` , `ScriptImage` , the diagnostic event tables, the audit event tables) carry the majority of the row count and growth rate. The remaining tables are stable in size or grow slowly enough that the existing maintenance pattern handles them comfortably.

The challenge with the high-volume tables is not that they are large today. It is that the operational cost of working with them grows faster than linearly as they continue to grow. Three specific costs become visible.

Index maintenance cost. Rebuilding or reorganizing an index on a five hundred million row table is an order of magnitude more expensive than the same operation on a fifty million row table. The work is not strictly linear because the page chain is longer and the locking and logging behavior is different. The nightly maintenance window contains the index work, and the window does not have unbounded room to grow.

Statistics quality. Statistics on very large tables either get sampled (which trades quality for collection time) or get expensive to update. A poorly sampled statistic on a high cardinality column can lead the optimizer to a bad plan choice on a query that previously had a good plan. The plan instability we already see in the cross-MFC Query Store evidence is partly an index coverage problem and partly a statistics quality problem.

Bulk delete cost. This is the cost the maintenance plan addresses through partition switching, but it is worth restating here. Iterative block deletes scale with the number of rows being deleted. Partition switches scale with the number of partitions being swapped, which is constant regardless of the row count inside the partition. The asymptotic behavior is fundamentally different.

The growth picture is not catastrophic in the short term. The architecture has held up at fourteen sites for several years and the daily operational rhythm is stable. The concern is forward looking: at twenty sites, at thirty sites, with two more years of data accumulation, the same architecture will not produce the same outcomes. We can either invest in the architecture now while there is operational room to plan and stage the work, or we can retrofit it under pressure later. The roadmap below assumes we are making the investment now.

3. The Partitioning POC

The partitioning proof of concept is a five-module body of work covering the essentials of bringing partitioning to PharmAssist:

- **Module 1.** What partitioning is, why it matters for high-volume OLTP, and the criteria for selecting candidate tables.
- **Module 1b.** The migration approach for moving an existing populated table to a partitioned layout.
- **Module 2.** The indexing rules for partitioned tables (aligned indexes, unaligned indexes, the role of the partition key).
- **Module 3.** The partition switch operation patterns that integrate with the maintenance procedures.
- **Module 4.** The partition lifecycle management (sliding window: split forward, switch out the oldest, merge as needed).
- **Module 5.** How we confirm the partitioning is doing what we expect.

The POC is complete. It has been exercised against representative data shapes and the migration approach has been validated end to end on a controlled subset. The next step is graduating it from a controlled environment to a production rollout. The full module text is available on request.

4. Rollout Plan

The rollout is staged. Partitioning a busy production table is not a lift that we want to do all at once across all candidate tables in a single change window. The plan is a sequence of single-table rollouts, each of which fits inside a normal change window, and each of which produces a partitioned table that the maintenance procedures can immediately benefit from.

4.1 Candidate Selection

The candidate tables, ranked by expected payback, are listed in Module 1. The short list:

- `EvtEvent` . Highest event-rate table. Sliding window retention is a clean fit.
- `0eOrderHistory` . Highest read pressure (the procedure analyses repeatedly cite this table). Partitioning by `HistoryDtTm` aligns with the dominant access pattern.
- `0eScriptImage` . LOB heavy. Partition aware retention also addresses the LOB page count growth issue.
- `EvtDiagEvent` and the related diagnostic event tables. Lower read pressure but high write rate. Partition switches collapse the purge window.
- `EvtInvEventLog` . Inventory audit log. High write rate, lower read pressure.

The full list with rowcounts and growth rates is in Module 1 and will be refreshed alongside the next quarterly capture.

4.2 Pilot Table

The first table to be partitioned in production is the smallest of the candidates with the highest payback ratio. It is deliberately not the highest-volume table. The highest-volume table is the wrong place to learn the production migration on. The pilot is the dry run that catches the operational issues we did not see in the POC environment, and once it has run for thirty days we know the rollout pattern works in production.

I will recommend the pilot table after the next quarterly capture so that the choice is informed by current growth rates rather than the February data shape.

4.3 Migration Approach

The migration approach is in Module 1b. The summary: a new partitioned table is created with the same schema, data is copied in batches from the existing table, the existing indexes are recreated as aligned indexes on the new table, the application is paused briefly, the tables are renamed, and the old table is retained for a rollback window before being dropped.

The migration is staged so that the application pause is the smallest possible window. On a table with hundreds of millions of rows, the data copy is the bulk of the elapsed time and runs concurrent with normal operations. The table rename is the only operation that requires the application pause, and it is a metadata operation that completes in seconds.

The rollback path is preserved through the retention window. If the partitioned table misbehaves in the first several days, we rename back to the original and lose nothing.

4.4 Application of Partition Switches in Maintenance

Once a candidate table is partitioned, the corresponding nightly purge procedure is modified to prefer `SWITCH PARTITION` to iterative deletes when the target is partitioned. The detection logic and the routing is described in section 5 of the Maintenance Operations Plan. This is the integration point where the scalability roadmap and the maintenance plan meet.

4.5 Sliding Window Lifecycle

After the migration, each partitioned table has a sliding window lifecycle. A new partition is created at the leading edge before the date threshold rolls over. The oldest partition is switched out at the trailing edge once it has aged past the retention window. The lifecycle runs as part of the nightly maintenance orchestration and is described in Module 4.

The retention window per table is a policy decision that needs operations and product input. The partitioning approach itself is policy agnostic. The retention can be sixty days, six months, or six years per table without changing the architecture.

4.6 Post-Migration Monitoring per MFC

A migration that has been executed at a site is not a migration closed at that site. The MFC DBA team and I monitor the partitioned table at each site for a defined post-migration window and confirm that the read workload, the write workload, the maintenance window, and the plan choices on procedures that touch the table are all behaving as the analysis predicted. The monitoring uses the Module 5 instrumentation and the same Query Store views that drive the procedure refactor program.

The data-shape variance that already exists across MFCs applies to partitioned tables as much as it applies to procedures. A partition layout, a partition function, and an aligned index set that perform well at the first site to be migrated may interact differently with a different data distribution at another site. Per-site monitoring is what catches that condition before it becomes operational. Where a site shows unexpected behavior after migration (a plan choice that did not match the prediction, a maintenance window that did not collapse as expected, a query whose partition elimination is not happening), we open a per-site root cause analysis. The output is either an explanation grounded in data shape, a refinement to the partitioning approach for that site, or an acknowledged exception. The same three-resolution framework that governs the procedure refactor program applies here.

5. Architectural Items Beyond Partitioning

The refactor program surfaced a small number of architectural changes that are too large to fit inside a procedure-level analysis. They are catalogued here so that they are visible and so that the engineering and architecture teams can evaluate them on their own merits.

5.1 Replace the PMSS Polling Pattern with Service Broker or Signal Based Handoff

`lsp_PmssIwebGetTopQueuedTrx` runs roughly 825 million times per month fleet wide. The cross-MFC view in the analysis for that procedure shows the call rate at the busiest sites is on the order of thirty to fifty calls per second. This is a tight polling loop that asks the database "is there work for me to do" and gets an answer of "no" the overwhelming majority of the time.

The right tier-one fix is the index addition described in the open items section of that analysis, which collapses the per-call cost of the poll from the current variable cost (with an 18x spread across plan variants at Tolleson alone) to single-digit reads per call. That fix is in flight as part of the refactor program.

The right tier-two fix is to replace the polling pattern with a handoff that does not require continuous database calls. Service Broker is the canonical SQL Server mechanism. A signal based pattern at the application layer (the IA Web Services interface waits on an event rather than polling) is an alternative if the application architecture supports it. Either approach removes the 825 million calls per month from the database load entirely.

This item requires application team coordination and is not in scope for the refactor program. It belongs in the scalability roadmap because it is an architectural shift, and it is the largest single architectural lever available against the current call volume.

5.2 Filtered Indexes on the OeGroup Hot Predicates

The Row 15 pilot identified that `OeGroup` (29 million rows, 4 GB clustered index) has zero index coverage for the filter predicates of `lsp_ShpGetOrdersForTopReadyToShipGroup` Part 1. The procedure-level fix is to add an index that covers the predicate. The architecture-level fix is to evaluate filtered indexes for the small set of recurring "in flight" status combinations and apply them across the procedures that share the predicate.

The filtered index approach is covered in the relevant masterclass entry. It needs DBA review because filtered indexes have to match query predicates exactly to be used, and the constraint applies fleet wide.

5.3 LOB Strategy on the Image Tables

`OeScriptImage`, `OeDocumentImage`, and the related image tables carry substantial LOB data that drives a large share of the storage footprint and the LOB page read load on the procedures that touch them. The `Img` procedure refactor (`lsp_ImgGetListOfTopXImagesToMove`) showed that removing a single LOB column predicate eliminated 100 percent of the LOB reads for that procedure. The pattern is general: predicates that touch LOB columns are expensive in their own right because LOB pointers have to be inspected.

The architecture level question is whether the LOB columns belong in the same table as the row metadata, or whether they should be split into a sibling table that the metadata table joins to when the LOB content is needed. The split would eliminate LOB page reads on procedures that only need metadata, at the cost of a join for procedures that need the content. The trade-off has to be evaluated per table because the access pattern varies.

This item is large, requires schema migration, and is not on the immediate roadmap. It is documented here so that it does not get lost.

5.4 Read Workload Separation

If we continue to add MFC sites, at some point the OLTP workload at a single instance will exceed what a single instance can comfortably handle even with all of the optimizations above. The standard SQL Server response is to add a read replica via Always On Availability Groups and route the read-heavy queries to the replica.

This is not a near-term need. It belongs on the roadmap so that we can evaluate the inflection point before it happens. The diagnostic data we are now collecting quarterly (Query Store, wait stats, data velocity) is exactly the input that informs the inflection point evaluation.

6. Risks and Dependencies

The largest risk in the program is the same one identified in the program charter: the deployment cadence gap. The partitioning rollout is a multi-team activity that depends on the iA DBA team, the iA software development teams, and the application teams that own the callers of the partitioning candidate tables. None of those teams report through me. The recommendation can be ready, the analysis can be complete, and the rollout still cannot proceed faster than the slowest of the involved teams can absorb the change. The mitigation is to keep the recommendation tightly scoped (one table at a time), to make the evidence and risk packaging as complete as possible, and to publish the candidate table choice early so that the involved teams have the maximum runway.

The second risk is the migration itself. The migration approach in Module 1b is the standard approach for moving a populated table to a partitioned layout, and the rollback path is preserved, but the migration on a busy production table is still a real operational event. The mitigation is the staged rollout (pilot first, smallest payback-positive table first), the rollback window, and the operations team's standard change management process.

The third risk is that an aligned index that performs well on the unpartitioned table performs differently on the partitioned table. Partitioned indexes are physically organized differently, and the optimizer's plan choices can shift. The Module 2 indexing strategy describes the alignment rules. The Module 5 monitoring confirms the plans are stable post-migration. Any procedure whose plan shifts unfavorably gets revisited.

The fourth risk is application contract. Most of the application code is partition-agnostic and will not need changes. The places where it is not partition-agnostic (cross-partition queries that do not provide a partition predicate, ad-hoc queries that scan whole tables) need to be identified by the iA development teams and either constrained or refactored. The refactor program is incidentally surfacing some of these as it walks the procedures, and the open items in those analyses point at the partitioning effort where relevant.

The fifth risk is the architectural items beyond partitioning. Service Broker rollout, filtered indexes, and LOB splitting all require coordination with application teams whose schedules are not under our control. None of them are blocking the partitioning rollout, and each can be evaluated and scheduled on its own timeline by the iA team and the application teams.

7. Phasing

The scalability roadmap maps to the program charter phasing as follows. The phases describe the analytical and recommendation work that the MFC DBA team and I produce. The actual production rollout cadence depends on the iA team's and the application teams' capacity to execute, and is not committed by these phases.

Phase 1 (Q2 2026): keep the POC current, refresh the candidate table list against the next quarterly capture, recommend the pilot table choice, and prepare the migration plan recommendation for the pilot. No production migrations are committed in this phase. Whether the iA and application teams choose to begin the pilot in Phase 1 or in a later phase is their decision, informed by the recommendation package.

Phase 2 (Q3 2026): if the iA and application teams begin the pilot in this phase, the MFC DBA team and I support the pilot with monitoring per Module 5, lessons-learned capture in our running log, and pattern-level findings written into the masterclass library. If the pilot is deferred, Phase 2 instead deepens the architectural recommendation backlog: the Service Broker conversion recommendation for the PMSS polling pattern, the filtered index recommendations, and the LOB strategy recommendation for the image tables. Either way, the analytical pace continues.

Phase 3 (Q4 2026 through Q1 2027): the remaining candidate-table migration recommendations are published in sequence, alongside the corresponding partition-switch recommendations for the maintenance procedures. The architectural recommendations from Phase 2 are evaluated by the iA team and the application teams against their roadmaps. The cataloged backlog at the end of Phase 3 should cover everything we have currently identified, organized in a form that the iA team can prioritize against their own plan.

The phasing is deliberately conservative on the analytical side. The point of holding to a steady analytical pace is that whenever the iA and application teams have bandwidth to execute, the recommendations are ready. A faster execution pace on the iA side is possible if the recommendations and evidence packages are accepted cleanly. A slower execution pace is the current norm and the program is designed to keep producing value at our pace regardless.

8. Expected Outcomes

Three outcomes from the scalability roadmap are worth calling out at the program level.

The first is that the per-table maintenance window stops being a function of the number of rows in the table. On a partitioned table with partition switches in place, the maintenance window is bounded by the number of partitions being swapped, regardless of the row count. This decouples maintenance time from data growth.

The second is that the read workload on the partitioned tables benefits from partition elimination automatically. Any query whose predicate aligns with the partition key reads only the relevant partitions. For the high-volume tables, where most queries are time-bounded, partition elimination reduces logical reads on those queries by a substantial multiple. The exact multiple depends on the partition count and the query selectivity. The Module 5 monitoring will quantify it post-migration on the pilot table.

The third is that the organization gains a path forward that is no longer pinned to the current architecture. With the diagnostic toolkit collecting data quarterly, the partitioning rollout proceeding on a known cadence, and the maintenance plan integrating the partition switch path as tables become eligible, the system is set up to absorb several years of growth without another firefighting cycle. That is the program-level definition of "scaling gracefully."

The combined effect of the four streams (refactor, maintenance, partitioning, and the architectural items) is the program charter's working target: support two to three times the current data volume on the same hardware with steady or improving response times.

9. How This Document Will Evolve

This roadmap is a living document. The candidate table list will be refreshed against each quarterly capture. The pilot table choice will be made and recorded once the next capture lands. The progress through the rollout will be logged in this document, with status updates appended rather than rewritten so that the history is preserved.

The architectural items in section 5 will be promoted to their own sections (or split into separate documents) as they progress. Items that are evaluated and rejected will remain in the document with the reasoning recorded, so that future evaluations do not have to redo the analysis.